

Supervised Learning

Classification, Regression, Structured Prediction

Classification / Regression:

$$f : \mathcal{X} \rightarrow \mathbb{N} \quad \text{or} \quad f : \mathcal{X} \rightarrow \mathbb{R}$$

- ▶ Inputs $x \in \mathcal{X}$ can be any kind of objects
 - ▶ images, text, audio, sequence of amino acids, ...
- ▶ Output $y \in \mathbb{N}/y \in \mathbb{R}$ is a discrete or real number
 - ▶ classification, regression, density estimation, ...

Structured Output Learning:

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

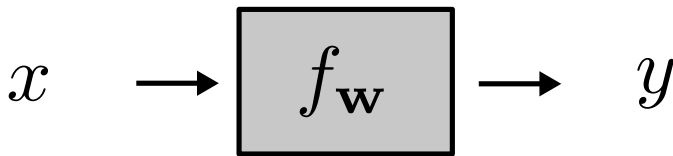
- ▶ Inputs $x \in \mathcal{X}$ can be any kind of objects
- ▶ Outputs $y \in \mathcal{Y}$ are complex (structured) objects
 - ▶ images, text, parse trees, folds of a protein, computer programs, ...

Supervised Learning

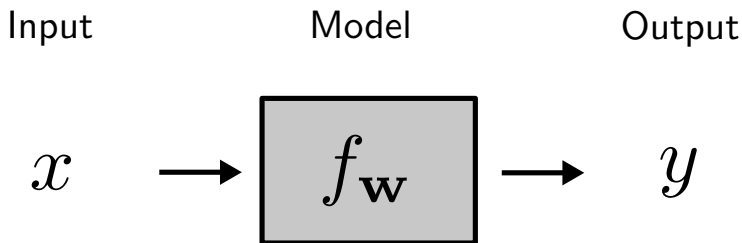
Input

Model

Output

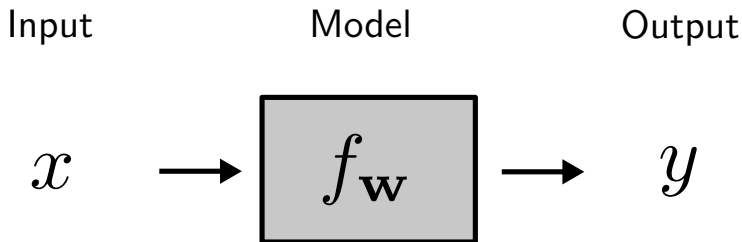


Supervised Learning



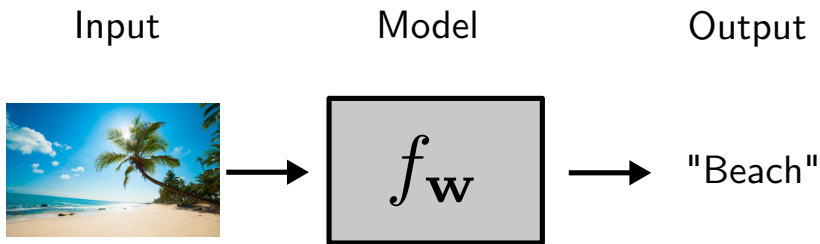
- **Learning:** Estimate parameters $\mathbf{w}$ from training data $\{(x_i, y_i)\}_{i=1}^N$

Supervised Learning



- **Learning:** Estimate parameters $\mathbf{w}$ from training data $\{(x_i, y_i)\}_{i=1}^N$
- **Inference:** Make novel predictions: $y = f_{\mathbf{w}}(x)$

Classification



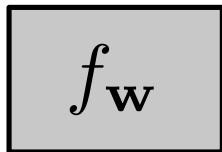
► **Mapping:** $f_{\mathbf{w}} : \mathbb{R}^{W \times H} \rightarrow \{\text{"Beach"}, \text{"No Beach"}\}$

Regression

Input



Model

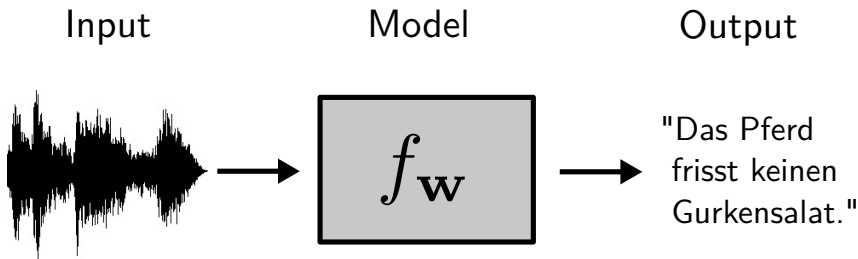


Output

143,52 €

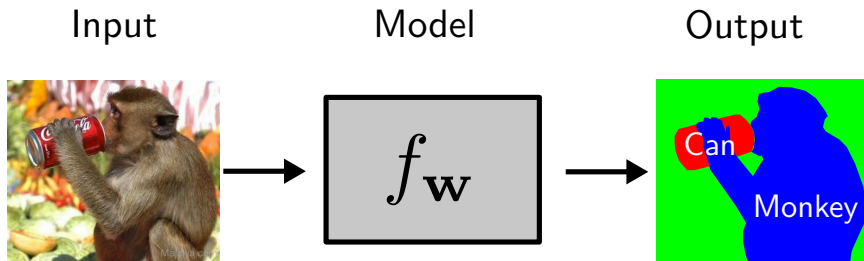
► Mapping: $f_{\mathbf{w}} : \mathbb{R}^N \rightarrow \mathbb{R}$

Structured Prediction



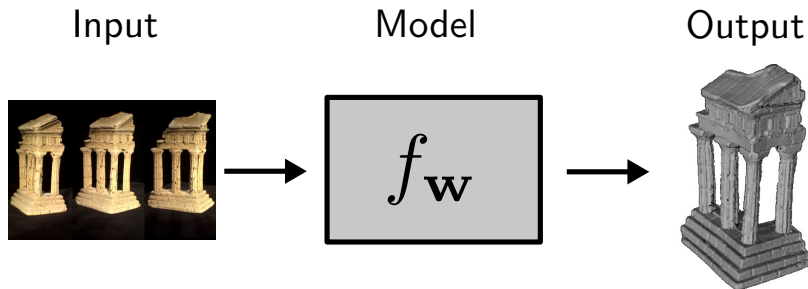
► **Mapping:** $f_{\mathbf{w}} : \mathbb{R}^N \rightarrow \{1, \dots, C\}^M$

Structured Prediction



► **Mapping:** $f_{\mathbf{w}} : \mathbb{R}^{W \times H} \rightarrow \{1, \dots, C\}^{W \times H}$

Structured Prediction



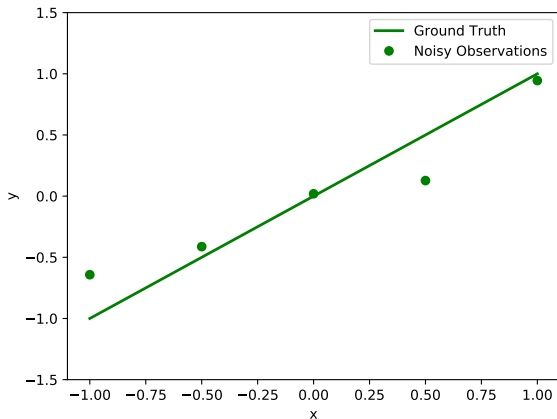
- **Mapping:** $f_{\mathbf{w}} : \mathbb{R}^{W \times H \times N} \rightarrow \{0, 1\}^{M^3}$
- **Suppose:** 32^3 voxels, binary variable per voxel (occupied/free)
- **Question:** How many different reconstructions? $2^{32^3} = 2^{32768}$
- **Comparison:** Number of atoms in the universe? $\sim 2^{273}$

Linear Regression

Linear Regression

Let $\mathcal{X}$ denote a dataset of size N and let $(\mathbf{x}_i, y_i) \in \mathcal{X}$ denote its elements ($y_i \in \mathbb{R}$).

Goal: Predict y for a previously unseen input $\mathbf{x}$. The input $\mathbf{x}$ may be multidimensional.

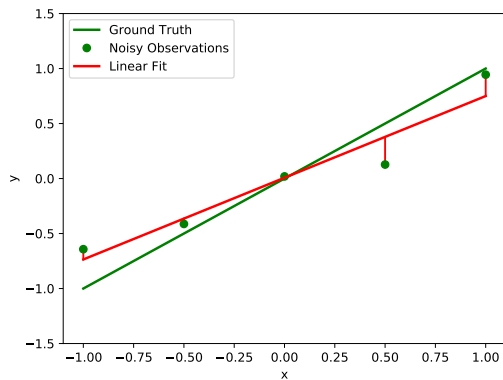


Linear Regression

The **error function** $E(\mathbf{w})$ measures the displacement along the y dimension between the data points (green) and the model $f(\mathbf{x}, \mathbf{w})$ (red) specified by the parameters $\mathbf{w}$.

$$f(\mathbf{x}, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}$$

$$\begin{aligned} E(\mathbf{w}) &= \sum_{i=1}^N (f(\mathbf{x}_i, \mathbf{w}) - y_i)^2 \\ &= \sum_{i=1}^N (\mathbf{x}_i^\top \mathbf{w} - y_i)^2 \\ &= \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \end{aligned}$$



Here: $\mathbf{x} = [1, x]^\top \Rightarrow f(\mathbf{x}, \mathbf{w}) = w_0 + w_1 x$

Linear Regression

The **gradient of the error function** with respect to the parameters $\mathbf{w}$ is given by:

$$\begin{aligned}\nabla_{\mathbf{w}} E(\mathbf{w}) &= \nabla_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \\ &= \nabla_{\mathbf{w}} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= \nabla_{\mathbf{w}} \left(\mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y} \right) \\ &= 2\mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{X}^\top \mathbf{y}\end{aligned}$$

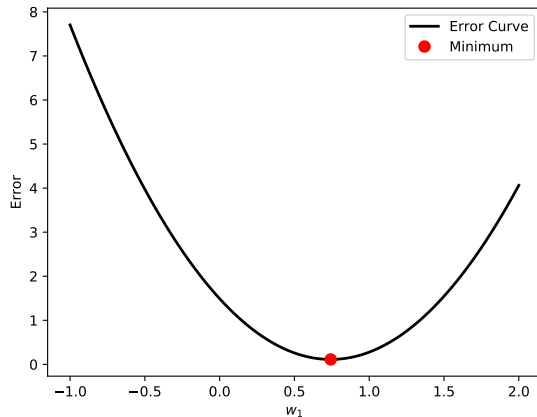
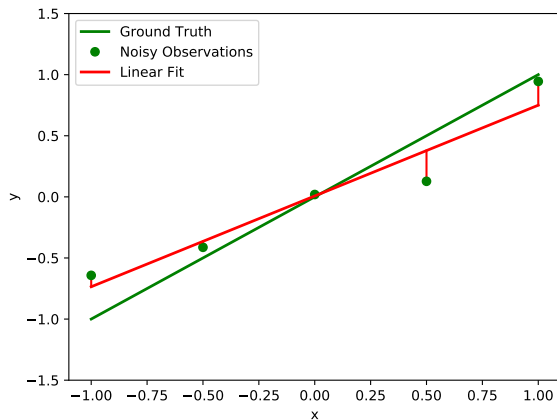
As $E(\mathbf{w})$ is quadratic and convex in $\mathbf{w}$, its minimizer (wrt. $\mathbf{w}$) is given in closed form:

$$\begin{aligned}\nabla_{\mathbf{w}} E(\mathbf{w}) &= 0 \\ \Rightarrow \mathbf{w} &= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}\end{aligned}$$

The matrix $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ is also called **Moore-Penrose inverse** or pseudoinverse.

Example: Line Fitting

Line Fitting



Linear least squares fit of model $f(\mathbf{x}, \mathbf{w}) = w_0 + w_1 x$ (red) to data points (green). Errors are also shown in red. Right: Error function $E(\mathbf{w})$ wrt. parameter w_1 .

Example: Polynomial Curve Fitting

Polynomial Curve Fitting

Let us choose a **polynomial of order M** to model dataset $\mathcal{X}$:

$$f(x, \mathbf{w}) = \sum_{j=0}^M w_j x^j = \mathbf{w}^\top \mathbf{x} \quad \text{with features} \quad \mathbf{x} = (1, x^1, x^2, \dots, x^M)^\top$$

Tasks:

- **Training:** Estimate $\mathbf{w}$ from dataset $\mathcal{X}$
- **Inference:** Predict y for novel x given estimated $\mathbf{w}$

Note:

- Features can be anything, including multi-dimensional inputs (e.g., images, audio), radial basis functions, sine/cosine functions, etc. In this example: monomials.

Polynomial Curve Fitting

Let us choose a **polynomial of order M** to model the dataset $\mathcal{X}$:

$$f(x, \mathbf{w}) = \sum_{j=0}^M w_j x^j = \mathbf{w}^\top \mathbf{x} \quad \text{with features} \quad \mathbf{x} = (1, x^1, x^2, \dots, x^M)^\top$$

How can we estimate $\mathbf{w}$ from $\mathcal{X}$?

Polynomial Curve Fitting

The **error function** from above is quadratic in $\mathbf{w}$ but not in x :

$$E(\mathbf{w}) = \sum_{i=1}^N (f(x_i, \mathbf{w}) - y_i)^2 = \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 = \sum_{i=1}^N \left(\sum_{j=0}^M w_j x_i^j - y_i \right)^2$$

It can be rewritten in the **matrix-vector notation** (i.e., as linear regression problem)

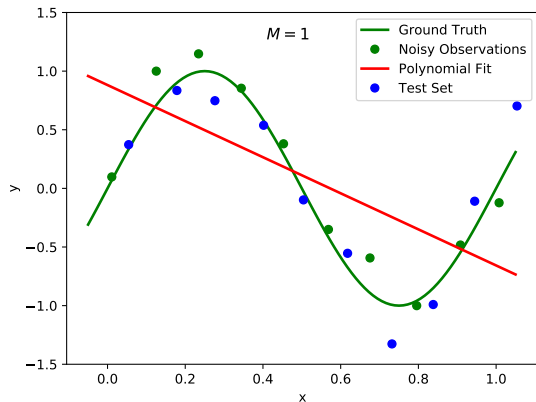
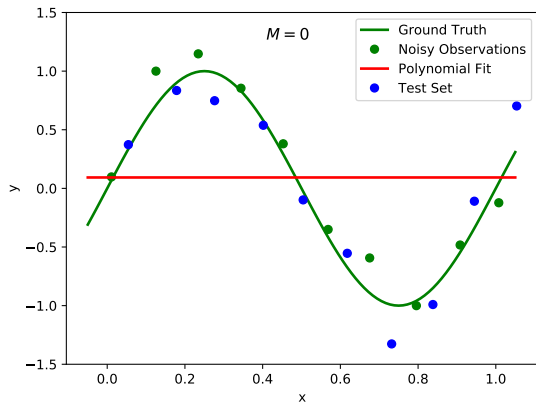
$$E(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

with feature matrix $\mathbf{X}$, observation vector $\mathbf{y}$ and weight vector $\mathbf{w}$:

$$\mathbf{X} = \begin{pmatrix} \vdots & \vdots & \vdots & & \vdots \\ 1 & x_i & x_i^2 & \dots & x_i^M \\ \vdots & \vdots & \vdots & & \vdots \end{pmatrix} \quad \mathbf{y} = \begin{pmatrix} \vdots \\ y_i \\ \vdots \end{pmatrix} \quad \mathbf{w} = \begin{pmatrix} w_0 \\ \vdots \\ w_M \end{pmatrix}$$

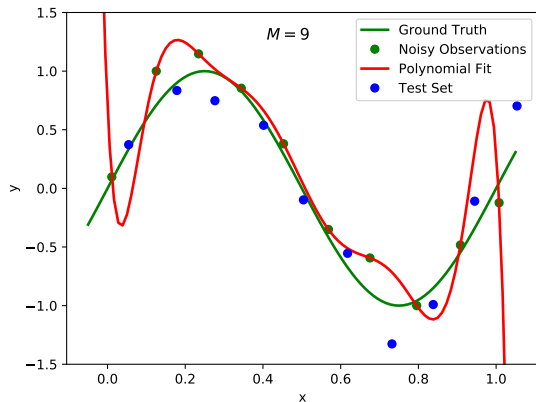
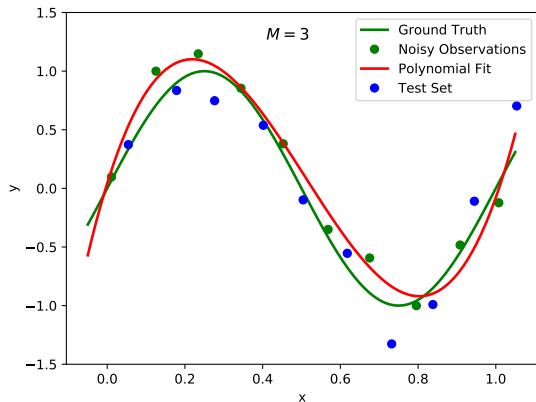
Polynomial Curve Fitting Results

Polynomial Curve Fitting



Plots of polynomials of various degrees M (red) fitted to the data (green). We observe underfitting ($M = 0/1$) and overfitting ($M = 9$). This is a model selection problem.

Polynomial Curve Fitting

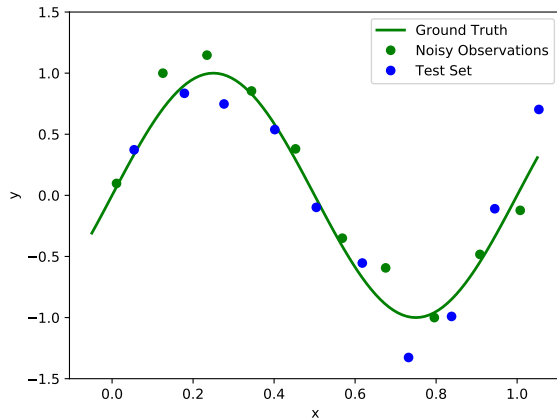


Plots of polynomials of various degrees M (red) fitted to the data (green). We observe underfitting ($M = 0/1$) and overfitting ($M = 9$). This is a model selection problem.

Capacity, Overfitting and Underfitting

Goal:

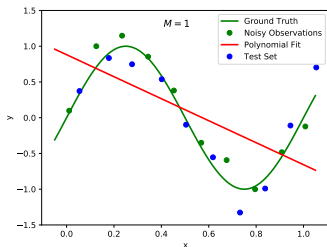
- ▶ Perform well on new, previously unseen inputs (test set, blue), not only on the training set (green)
- ▶ This is called **generalization** and separates ML from optimization
- ▶ Assumption: training and test data independent and identically (i.i.d.) drawn from distribution $p_{data}(x, y)$
- ▶ Here: $p_{data}(x) = \mathcal{U}(0, 1)$
 $p_{data}(y|x) = \mathcal{N}(\sin(2\pi x), \sigma)$



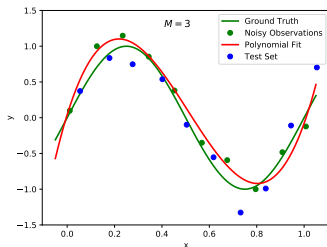
Capacity, Overfitting and Underfitting

Terminology:

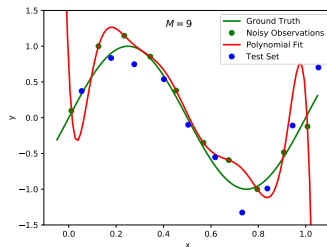
- **Capacity:** Complexity of functions which can be represented by model f
- **Underfitting:** Model too simple, does not achieve low error on training set
- **Overfitting:** Training error small, but test error (= generalization error) large



Capacity too low



Capacity about right

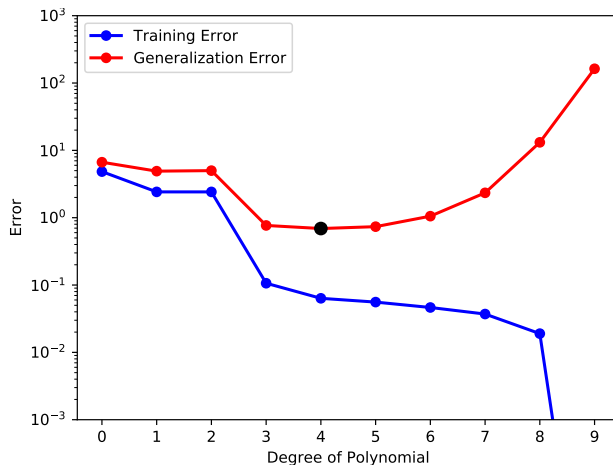


Capacity too high

Capacity, Overfitting and Underfitting

Example: Generalization error for various polynomial degrees M

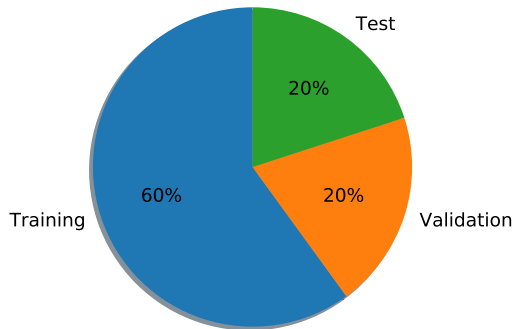
- Model selection: Select model with the smallest generalization error



Capacity, Overfitting and Underfitting

General Approach: Split dataset into training, validation and test set

- Choose hyperparameters (e.g., degree of polynomial, learning rate in neural net, ..) using validation set. Important: Evaluate once on test set (typically not available).



- When dataset is small, use (k-fold) cross validation instead of fixed split.

Ridge Regression

Ridge Regression

Polynomial Curve Model:

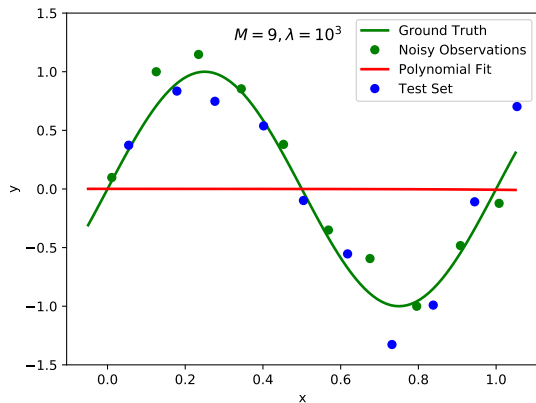
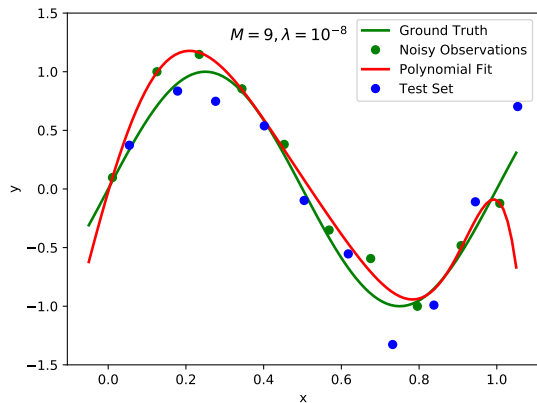
$$f(x, \mathbf{w}) = \sum_{j=0}^M w_j x^j = \mathbf{w}^\top \mathbf{x} \quad \text{with features} \quad \mathbf{x} = (1, x^1, x^2, \dots, x^M)^\top$$

Ridge Regression:

$$\begin{aligned} E(\mathbf{w}) &= \sum_{i=1}^N (f(x_i, \mathbf{w}) - y_i)^2 + \lambda \sum_{j=0}^M w^2 \\ &= \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2 \end{aligned}$$

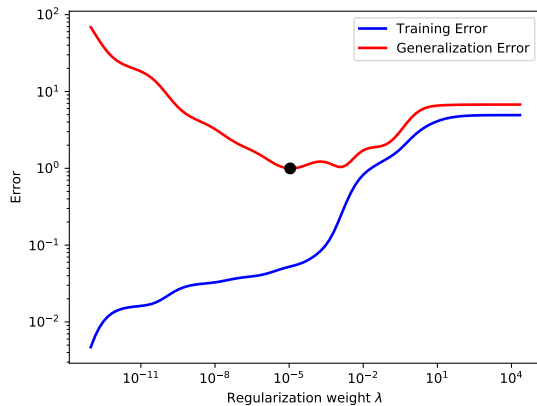
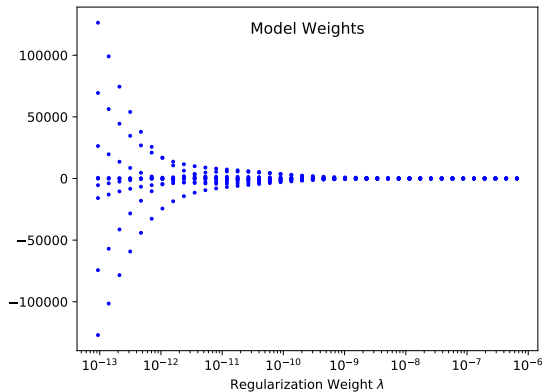
- Idea: Discourage large parameters by adding a regularization term with strength λ
- Closed form solution: $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$

Ridge Regression



Plots of polynomial with degree $M = 9$ fitted to 10 data points using ridge regression. Left: weak regularization ($\lambda = 10^{-8}$). Right: strong regularization (right, $\lambda = 10^3$).

Ridge Regression



Left: With low regularization, parameters can become very large (ill-conditioning).

Right: Select model with the smallest generalization error on the validation set.

Estimators, Bias and Variance

Estimators, Bias and Variance

Point Estimator:

- ▶ A point estimator $g(\cdot)$ is function that maps a dataset $\mathcal{X}$ to model parameters $\hat{\mathbf{w}}$:

$$\hat{\mathbf{w}} = g(\mathcal{X})$$

- ▶ Example: Estimator of ridge regression model: $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$
- ▶ We use the hat notation to denote that $\hat{\mathbf{w}}$ is an estimate
- ▶ A good estimator is a function that returns a parameter set close to the true one
- ▶ The data $\mathcal{X} = \{(x_i, y_i)\}$ is drawn from a random process $(x_i, y_i) \sim p_{data}(\cdot)$
- ▶ Thus, any function of the data is random and $\hat{\mathbf{w}}$ is a random variable.

Estimators, Bias and Variance

Properties of Point Estimators:

Bias:

$$\text{Bias}(\hat{\mathbf{w}}) = \mathbb{E}(\hat{\mathbf{w}}) - \mathbf{w}$$

- ▶ Expectation over datasets $\mathcal{X}$
- ▶ $\hat{\mathbf{w}}$ is unbiased $\Leftrightarrow \text{Bias}(\hat{\mathbf{w}}) = 0$
- ▶ A good estimator has little bias

Variance:

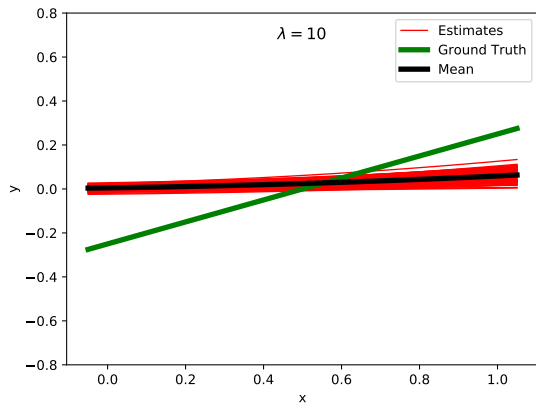
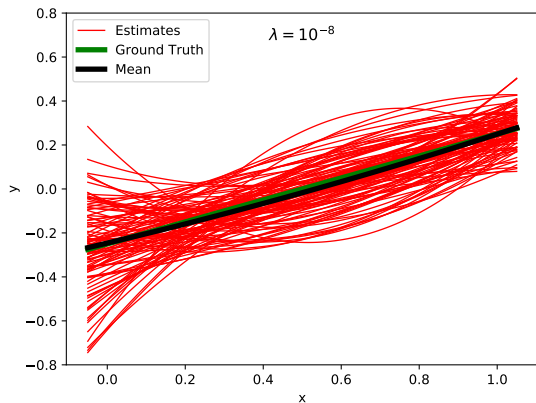
$$\text{Var}(\hat{\mathbf{w}}) = \mathbb{E}(\hat{\mathbf{w}}^2) - \mathbb{E}(\hat{\mathbf{w}})^2$$

- ▶ Variance over datasets $\mathcal{X}$
- ▶ $\sqrt{\text{Var}(\hat{\mathbf{w}})}$ is called “standard error”
- ▶ A good estimator has low variance

Bias-Variance Dilemma:

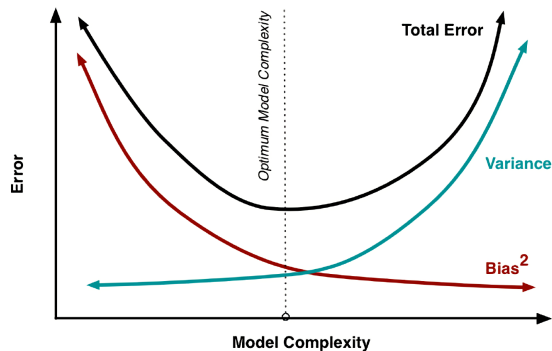
- ▶ Statistical learning theory tells us that we can't have both $\Rightarrow$ there is a trade-off

Estimators, Bias and Variance

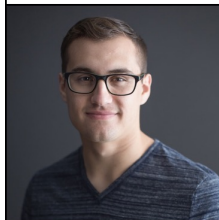


Ridge regression with weak ($\lambda = 10^{-8}$) and strong ($\lambda = 10$) regularization.
Green: True model. Black: Plot of model with mean parameters $\bar{\mathbf{w}} = \mathbb{E}(\mathbf{w})$.

Estimators, Bias and Variance



Statisticians Hate Him



Get low bias AND
LOW VARIANCE
with this one
WEIRD trick

LEARN THE TRUTH NOW

- ▶ There is a bias-variance tradeoff: $\mathbb{E}[(\hat{\mathbf{w}} - \mathbf{w})^2] = \text{Bias}(\hat{\mathbf{w}})^2 + \text{Var}(\hat{\mathbf{w}})$
- ▶ Or not? In deep neural networks the test error decreases with network width!
<https://www.bradyn Neal.com/bias-variance-tradeoff-textbooks-update>

Maximum Likelihood Estimation

Maximum Likelihood Estimation

- ▶ We now reinterpret our results by taking a **probabilistic viewpoint**
- ▶ Let $\mathcal{X} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ be a dataset with samples drawn i.i.d. from p_{data}
- ▶ Let the model $p_{model}(y|\mathbf{x}, \mathbf{w})$ be a parametric family of probability distributions
- ▶ The conditional **maximum likelihood estimator** for $\mathbf{w}$ is given by

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \operatorname{argmax}_{\mathbf{w}} p_{model}(\mathbf{y}|\mathbf{X}, \mathbf{w}) \\ &\stackrel{\text{iid}}{=} \operatorname{argmax}_{\mathbf{w}} \prod_{i=1}^N p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \operatorname{argmax}_{\mathbf{w}} \underbrace{\sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w})}_{\text{Log-Likelihood}}\end{aligned}$$

Note: In this lecture we consider log to be the natural logarithm.

Maximum Likelihood Estimation

Example: Assuming $p_{model}(y|\mathbf{x}, \mathbf{w}) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x}, \sigma)$, we obtain

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\&= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(\mathbf{w}^\top \mathbf{x}_i - y_i)^2} \right] \\&= \operatorname{argmax}_{\mathbf{w}} - \sum_{i=1}^N \frac{1}{2} \log(2\pi\sigma^2) - \sum_{i=1}^N \frac{1}{2\sigma^2} (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 \\&= \operatorname{argmax}_{\mathbf{w}} - \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 \\&= \operatorname{argmin}_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2\end{aligned}$$

Maximum Likelihood Estimation

We see that choosing $p_{model}(y|\mathbf{x}, \mathbf{w})$ to be Gaussian causes maximum likelihood to yield exactly the same least squares estimator derived before:

$$\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

Variations:

- ▶ If we were choosing $p_{model}(y|\mathbf{x}, \mathbf{w})$ as a Laplace distribution, we would obtain an estimator that minimizes the ℓ_1 norm: $\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_1$
- ▶ Assuming a Gaussian distribution over the parameters $\mathbf{w}$ and performing a maximum a-posteriori (MAP) estimation yields ridge regression:

$$\operatorname{argmax}_{\mathbf{w}} p(\mathbf{w}|y, \mathbf{x}) = \operatorname{argmax}_{\mathbf{w}} p(y|\mathbf{x}, \mathbf{w})p(\mathbf{w})$$

Maximum Likelihood Estimation

We see that choosing $p_{model}(y|\mathbf{x}, \mathbf{w})$ to be Gaussian causes maximum likelihood to yield exactly the same least squares estimator derived before:

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

Remarks:

- ▶ **Consistency:** As the number of training samples approaches infinity $N \rightarrow \infty$, the maximum likelihood (ML) estimate converges to the true parameters
- ▶ **Efficiency:** The ML estimate converges most quickly as N increases
- ▶ These theoretical considerations make ML estimators appealing